

```
// Code-A5 (N-Queen)
```

```
#include <iostream>
```

```
#include <vector>
```

```
using namespace std;
```

```
// Function to print the board
```

```
void printBoard(vector<vector<int>>& board, int n) {
```

```
    for (int i = 0; i < n; i++) {
```

```
        for (int j = 0; j < n; j++)
```

```
            cout << board[i][j] << " ";
```

```
        cout << endl;
```

```
    }
```

```
    cout << endl;
```

```
}
```

```
// Function to check if placing a queen is safe
```

```
bool isSafe(vector<vector<int>>& board, int row, int col, int n) {
```

```
    // Check column
```

```
    for (int i = 0; i < row; i++)
```

```
        if (board[i][col]) return false;
```

```
    // Check upper left diagonal
```

```
    for (int i = row - 1, j = col - 1; i >= 0 && j >= 0; i--, j--)
```

```
        if (board[i][j]) return false;
```

```
    // Check upper right diagonal
```

```
    for (int i = row - 1, j = col + 1; i >= 0 && j < n; i--, j++)
```

```
        if (board[i][j]) return false;
```

```
    return true;
```

```
}
```

```
// Backtracking function
```

```
bool solveNQueens(vector<vector<int>>& board, int row, int n) {
```

```
    if (row == n) {
```

```
        printBoard(board, n); // Print one valid arrangement
```

```
        return true;
```

```
    }
```

```
    for (int col = 0; col < n; col++) {
```

```
        if (isSafe(board, row, col, n)) {
```

```
            board[row][col] = 1; // Place queen
```

```
            solveNQueens(board, row + 1, n);
```

```
            board[row][col] = 0; // Backtrack
```

```
        }
```

```
    }
```

```
    return false;
```

```
}
```

```
int main() {
```

```
    int n, firstRow, firstCol;
```

```
    cout << "Enter size of board (N): ";
```

```
    cin >> n;
```

```
    vector<vector<int>> board(n, vector<int>(n, 0));
```

```
    cout << "Enter position of first queen (row and column index starting from 0): ";
```

```
    cin >> firstRow >> firstCol;
```

```
    // Place first queen
```

```
    board[firstRow][firstCol] = 1;
```

```

cout << "\nAll possible solutions:\n";

solveNQueens(board, 0, n); // Start solving from row 0


return 0;
}


// SAMPLE OUTPUT
/*
* $ ./a.out
* Enter size of board (N): 4
* Enter position of first queen (row and column index starting from 0): 0 0
*
* All possible solutions:
* 0 1 0 0
* 0 0 0 1
* 1 0 0 0
* 0 0 1 0
*
* 0 0 1 0
* 1 0 0 0
* 0 0 0 1
* 0 1 0 0
*/

```